

What is Hypertrace?

Hypertrace is an open source distributed tracing and observability platform that helps to make sense of the production request. It converts data into relevant insights for everyone. It helps the infrastructure team to identify which services are causing overload, allows the application team to understand which requests put their service objectives at risk, and lets the deployment team find the problems.

Why should you choose Hypertrace?

From extensibility and interoperability to extremely effective logs and alerts, Hypertrace has several features that make it a good choice for observability. Here is a quick look at some of the key features of this platform.

- **Topology:** Hypertrace lets you view the topology of all services and backends in real-time. It is scalable for hundreds of them.
- **Dashboards:** It provides you pre-scanned dashboards for services, APIs, and backends.
- **Extensibility:** You can write your own enrichers to enrich trace data for your business needs and create a view around them.
- **Interoperability:** Hypertrace works out-of-the-box with all open source tracing formats like Zipkin, OTel and Jaeger.
- **GraphQL APIs:** It traces, spans, and exposes entities through GraphQL APIs. It helps build the next creative use case.
- **API and trace analytics:** It helps to slice and dice all data powered by Apache Pinot.
- **Span logs:** Hypertrace speeds up root cause analysis (RCA) and reduces mean time to detect (MTTD) with span logs on the platform.
- **Error analytics:** It lets you find issues quickly with error highlighting and filters.
- **Alerting:** You get an alert on any deviation/anomalies in the out-of-

Visibility into dependent services

The old way: Whenever something broke, say, you found out that service X had some latency issues, you had it checked with the service owner to understand dependencies. Then you also looked at metrics that were exported to Prometheus manually.

The new way: In the new way, the dependency graph is already created with real-time metrics based on the available data.

Finding hotspots

The old way: Multiple dashboards were required to correlate.

The new way: Complete request is visualised in a single pane.

Log fatigue

The old way: You had to query logs by requestID and dig through all the logs generated across all the services for this request. The number of log lines also increased based on the number of dependencies.

The new way: You just figure out the problematic span and find correlated logs in the same view. If a particular span is taking more time than was accounted for, then you can click on it and see correlated logs in the context.

Debugging slow queries

The old way: There was a dependency on DBAs to get slow query logs. You had to manually log slow queries into the logging platform and create dashboards.

The new way: 'Automagic' creation of query dashboards can be done today.

Hypertrace deployment

From the infrastructure end of things you can easily deploy Hypertrace on any of the container environments like Kubernetes, EKS, ECS, AKS, VMs, GKE or Docker. It is compatible with Zipkin, OpenTelemetry and Jaeger. Language modules such as Java, Python and Go are also supported (refer to Figure 1).

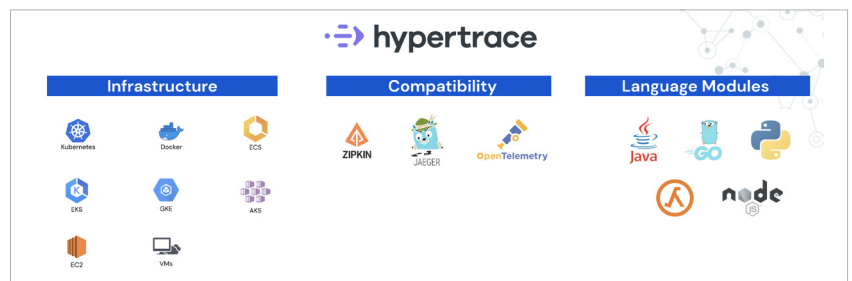


Figure 1: Infrastructure, compatibility and language modules supported by Hypertrace

bag (OOB) metrics. A host of such desirable features makes Hypertrace a great cloud-native distributed tracing based

tool for observability. It is evolving tremendously as people are continuously contributing towards it. Definitely something to check out! **END** 🐧

By: Jayesh Babu Ahire

The author is founding engineer/product manager at TraceableAI. This article is based on a tech talk he gave at OSI 2021, a virtual event.

Transcribed and curated by Laveesh Kocher